

Python 应用开发基础

课程报告

题目： 基于 YOLOv5 的口罩检测系统

组号： 第 13 组

成员： 吴钟扬、陈温特、吴同宇、许翔宇

2021 年 12 月 28 日

目 录

1	研究背景及意义	1
2	卷积神经网络发展历程	1
3	目标检测算法分析	2
3.1	传统目标检测算法.....	2
3.2	深度学习检测算法.....	2
3.2.1	Two-stage 目标检测算法.....	2
3.2.2	One-Stage 目标检测算法.....	2
4	YOLO 的分析和其演变	3
4.1	YOLOv1	3
4.2	YOLOv2	3
4.3	YOLOv3	4
4.4	YOLOv4	5
4.5	YOLOv5	5
4.5.1	输入端.....	5
4.5.2	YOLOv5 网络	6
5	核心工作	6
5.1	环境配置.....	6
5.1.1	硬件配置.....	6
5.1.2	软件配置.....	6
5.2	数据集的获取与处理.....	6
5.2.1	数据集获取.....	6

5.2.2	数据集标注.....	7
5.3	模型的训练.....	7
5.3.1	模型训练过程介绍.....	7
5.3.2	通过 K-means 算法生成合适的先验框	8
5.3.3	模型训练过程分析.....	9
5.4	模型训练结果的评估.....	10
5.5	模型的优化.....	12
5.6	成果展示.....	12
5.6.1	图片识别功能.....	12
5.6.2	视频和摄像头实时识别功能.....	13
6	总结与展望	14
6.1.1	总结.....	14
6.1.2	展望.....	14
6.1.3	感想.....	14

1 研究背景及意义

2019 年末爆发的 COVID-19，对全球造成了巨大的冲击。下图是截止 2021 年 12 月 28 日的的数据。

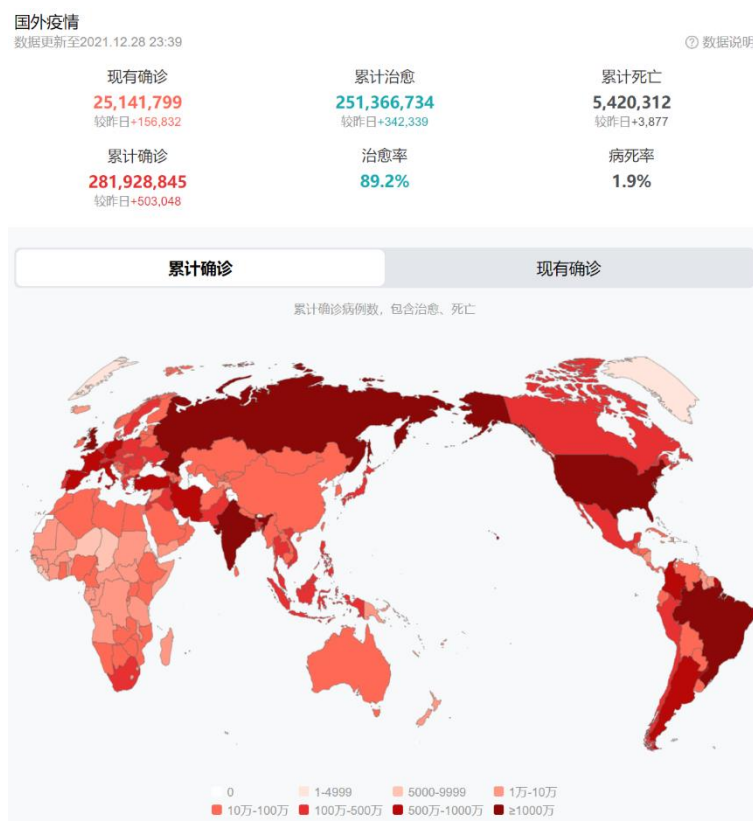


图 1 全球疫情数据（截至 2021.12.28）

此次疫情之下，人工智能在各行各业迅速崛起，为人类做出了许多突出的贡献，AI 无接触测温，AI 辅助诊断，无人机巡视等。预防新冠肺炎的方式主要是佩戴口罩，来阻断病毒在空气中的传播。基于这样的背景，我们实现了一个通过深度学习检测人们是否戴口罩的系统，并凭借实时摄像头或现有的图片和视频对范围内的人群进行检测，判断其是否佩戴口罩，从而及时提醒未戴口罩的人。这种方式能够有效地减少病毒的传播。

2 卷积神经网络发展历程

早期的单层神经网络在解决复杂的问题上效果不是很好，学习能力较差，而要增加学习能力核心就是要增加神经网络的结构。网络的层次越多，深度学习效果也会逐步提升，深度学习能够从复杂抽象的数据中提取有效信息，多层的神经网络的学习能力比单层神经网络更强，这也意味着它需要更强大的算法来训练。

卷积图像层的神经网络技术在图像处理领域中有一定的知名度，特别是对图像的分析表现尤其优秀。传统的图像分析和处理的算法往往需要人工的参与来完成图像预处理的过程，而卷积的神经网络在图像层的处理功能和其形式上已经做了一系列的变化，图像的预处理不再需要人工去筛选或改动，神经网络可以直接对输入的图片进行一系列的操作，大大减少了

人工的参与，至今已被广泛用于各种图像相关的应用中。

3 目标检测算法分析

3.1 传统目标检测算法

传统图像目标检测算法提取的特征主要集中于底层特征和中层次特征，例如颜色，纹理等。这类算法使用人工设计特征以及机器学习的分类方式，但它提取到的特征局限性较大且学习速度有限。

3.2 深度学习检测算法

目标检测算法的迅猛发展，得益于深度学习的广泛实践。衡量一个模型的性能，准确度和速度公认为两个指标，目前主流的深度学习目标检测算法有：One-Stage 和 Two-Stage，这两者都属于端到端的目标检测模型，且各有优劣。从准确度而言，Two-Stage 的性能优于 One-Stage；而在速度上，One-Stage 占有明显的优势。随着深度学习算法的不断发展，两类算法各取优点，能够很好地结合，不断提升算法的性能。

3.2.1 Two-stage 目标检测算法

Two-Stage 会进行两次分类，相当于运行了两次 One-Stage，先找到物体位置再进一步对每个候选区域进行精细监测，需要多次运行监测和分类的流程，所以 Two-Stage 算法相较于 One-Stage 算法性能要差。在实际应用的算法要比示例的简单网络复杂，会有残差结构等。R-CNN 是常见的可用于快速构建，并且精确程度相当高的目标检测模型：

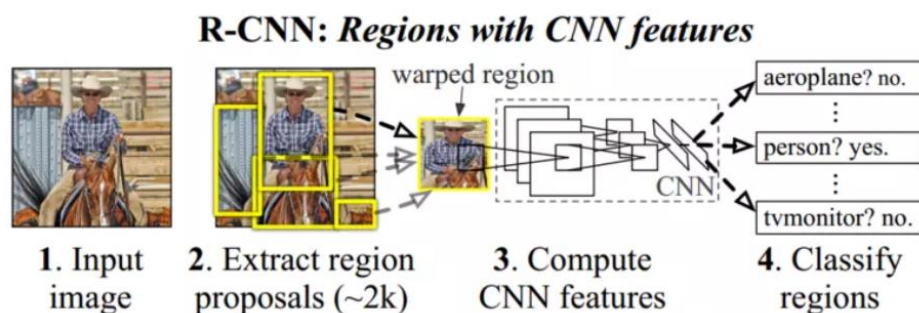


图 2 R-CNN

这类 Two-Stage 方法解决了传统目标检测算法的问题，在精度上有很大突破，但在速度上很难达到实时检测的效果。

3.2.2 One-Stage 目标检测算法

One-stage 目标检测算法特点是一步到位，速度相对较快，达到了实时级别的突破。我们使用的 YOLO 就是属于 One-stage 算法，YOLO 的早期版本虽然在准确率上有所欠缺，但到 v5 版本时准确率提高了很多。

4 YOLO 的分析和其演变

YOLO 将整张图像作为网络的输入，所以能够很好地提取全局信息，提取特征后直接分类与特征回归，然后就可以得到最终的检测结果了。相对于需要多次运行检测和分类流程的 Two-stage 算法，YOLO 速度相对较快，甚至可以实现对视频的实时检测。

4.1 YOLOv1

YOLO 将目标检测作为单个回归问题，直接使用一个卷积神经网络就可以得知边界框的位置和类别^{错误:未找到引用源。}。网络结构用 1×1 进行降维和 3×3 进行卷积。整个网络由 24 个卷积层和 2 个全连接层组成^{错误:未找到引用源。}，由于检测需要更多图片细节的信息，因此在训练检测网络时，将输入图片的大小从 244×244 调整为 448×448 ，最后使用全连接层进行类别输出。以下是 YOLOv1 的网络结构：

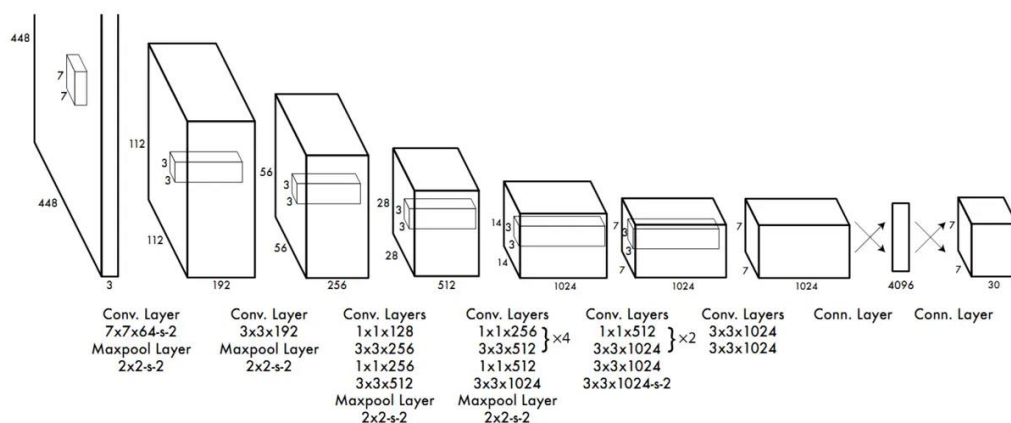


图 3 YOLOv1 网络结构

4.2 YOLOv2

YOLOv2 主要改进是提高了召回率和定位能力，移除全连接层和一个池化层，并且调整网络的输入由 448×448 的分辨率微调最初的 416×416 分类网络以保证特征图能够产生一个中心网格。输入尺寸大小会影响训练速度：输入尺寸较大时，训练速度较慢；而输入尺寸较小时，训练速度会快。训练时每迭代 10 次，就会微调分辨率大小，这样就可以适应多种不同尺度的输入，缓解从分类网络突然切换到检测网络时分辨率变化太大造成的影响。

借鉴 Faster R-CNN，YOLOv2 采用 k-means 聚类算法寻找合适的先验框，减少训练时微调分辨率到最终值的难度。由于欧氏距离的缘故，大边框会比小边框产生更大的误差，导致结果偏离。为了减小误差，使用 IOU 参与距离计算，因为 IOU 与边界框尺寸无关，这样计算后的 IOU 值比原先手动设置先验框的方法要高。

YOLOv2 提出 Darknet-19，使用池化层替代全连接做预测分类，而且每一个卷积层中都加入了批标准化(batch normalization)^{错误:未找到引用源。}，能够解决反向传播时梯度消失问题，提高稳定性，加速收敛，对模型正则化，也提升了分类正确率^{错误:未找到引用源。}。

Type	Filters	Size/Stride	Output
Convolutional	32	3×3	224×224
Maxpool		$2 \times 2/2$	112×112
Convolutional	64	3×3	112×112
Maxpool		$2 \times 2/2$	56×56
Convolutional	128	3×3	56×56
Convolutional	64	1×1	56×56
Convolutional	128	3×3	56×56
Maxpool		$2 \times 2/2$	28×28
Convolutional	256	3×3	28×28
Convolutional	128	1×1	28×28
Convolutional	256	3×3	28×28
Maxpool		$2 \times 2/2$	14×14
Convolutional	512	3×3	14×14
Convolutional	256	1×1	14×14
Convolutional	512	3×3	14×14
Convolutional	256	1×1	14×14
Convolutional	512	3×3	14×14
Maxpool		$2 \times 2/2$	7×7
Convolutional	1024	3×3	7×7
Convolutional	512	1×1	7×7
Convolutional	1024	3×3	7×7
Convolutional	512	1×1	7×7
Convolutional	1024	3×3	7×7
Convolutional	1000	1×1	7×7
Avgpool		Global	1000
Softmax			

图 4 Darknet-19 模型

4.3 YOLOv3

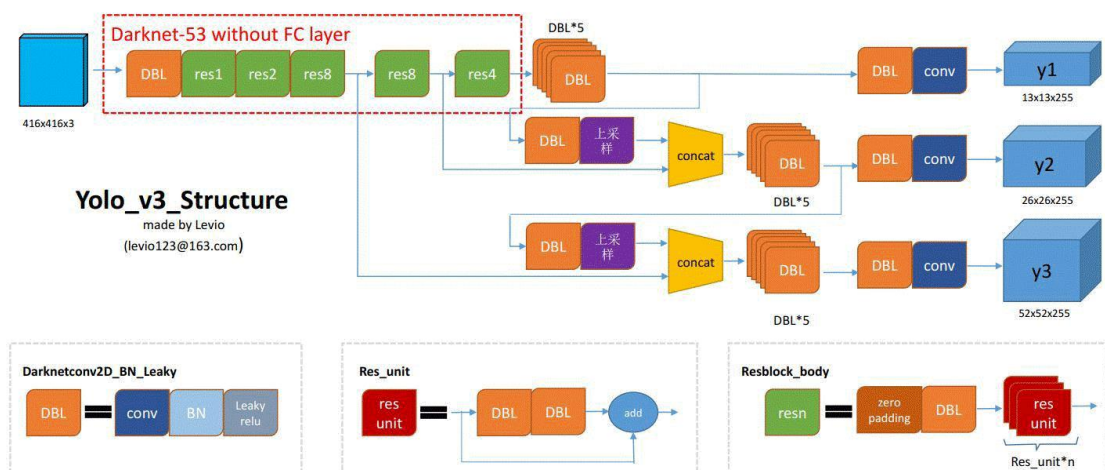


图 51 YOLOv3 模型

YOLOv3 算法的性能得到巨大提升得益于特征提取网络的增强,模型比之前的模型要复杂,在 YOLOv2 的基础上构造出更深的 DarkNet-53 网络。

YOLOv3 使用了残差网络,通常来讲,神经网络深度越深,是会引起梯度消失的,但是使用残参网络可以很好地解决这个问题,每个残参块都使用跳跃连接,即使层数已经到达很深的程度,也依旧能够保持高的准确率。为了降低池化带来的梯度负面效果,使用 5 个步长

4.5.2 YOLOv5 网络

YOLOv5 网络由三个主要组件组成：

- 1) Backbone: 在不同图像细粒度上聚合并形成图像特征的卷积神经网络。
- 2) Neck: 一系列混合和组合图像特征的网络层，并将图像特征传递到预测层。
- 3) output: 对图像特征进行预测，生成边界框和并预测类别。

5 核心工作

5.1 环境配置

5.1.1 硬件配置

- 1) CPU: Intel(R) Core(TM) i7-8750H CPU @ 2.20GHz
- 2) 显卡: NVIDIA GeForce RTX 2060
- 3) 操作系统: windows10
- 4) 内存: 16G

5.1.2 软件配置

- 1) Python: Python 3.8.0
- 2) 编辑器: PyCharm Community Edition 2021.3

5.2 数据集的获取与处理

5.2.1 数据集获取

首先是数据集的获取，我们采用的数据集来自多个方面，整合了通过网络渠道搜集而来的各种图片，且挑选了人的面部较为清晰的图片。

由于数据集的质量和数量也会影响到模型的训练结果，我们尽可能多地收集了数据。本次课程设计总共搜集了 1600 张图片，并将其分为了训练集（1200 张）、验证集和测试集（400 张）。以下是截取的部分图片数据集：

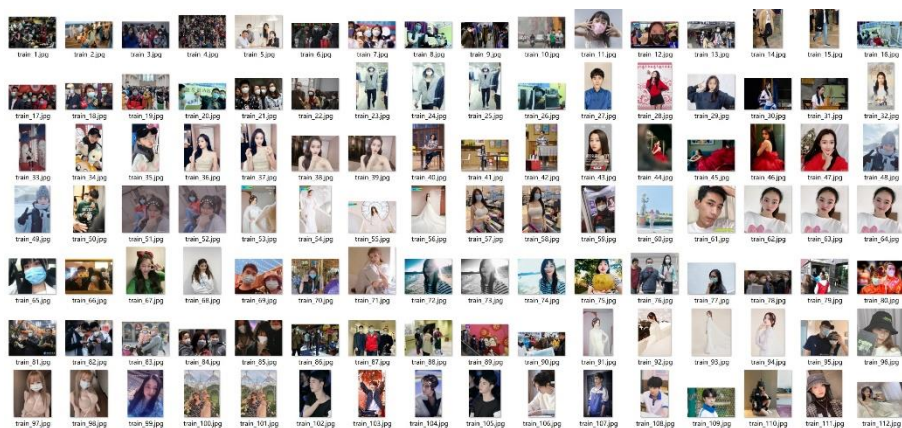


图 3 数据集（训练集）部分图片

在收集图片时，不需要特意选取某些尺寸的图片，因为在训练前，`train.py` 文件在读取训练图像和标签数据时会使用 `letterbox` 函数对图像和标签进行等比例缩放。包括检测时也会进行等比例缩放，所以我们无需手动对图片进行尺寸筛选或者改动。

5.2.2 数据集标注

接着我们需要对图片进行标注，我们使用了 `labelImage` 工具来标注图片集。人工标注时将整个脸部作为目标框标注出位置，并写入相应的数据标签 `mask` 或者 `face`。



图 8 使用 labelImage 的数据标注过程

5.3 模型的训练

5.3.1 模型训练过程介绍

我们使用的是 YOLOv5s 版本，识别的内容只有两类，只需要检测目标是佩戴了口罩还是没有佩戴口罩。创建 `mask_data.yaml`，并设置分类为 `[mask, face]`，下面是训练自己的训练集时的具体操作：

- 1) 将所有数据集图片存在 YOLO_MASK/score 目录下
- 2) 将 labelImage 标注后的标注文件分别存放在同一目录下的 labels 文件夹中
- 3) 在配置数据集相应的路径后，开始训练模型。

epoch	train/box_loss	train/obj_loss	train/cls_loss	metrics/precision	metrics/recall	metrics/mAP_0.5	metrics/mAP_0.5:0.95
0	0.098674	0.048647	0.026129	0.14449	0.33391	0.13021	0.023278
1	0.077256	0.037775	0.017227	0.2327	0.50611	0.19469	0.042202
2	0.073374	0.035053	0.0092267	0.34771	0.48038	0.37893	0.099927
3	0.06919	0.033951	0.0066651	0.49348	0.6765	0.52765	0.14251
4	0.056887	0.032866	0.0049626	0.60784	0.72832	0.62465	0.20372
5	0.053387	0.03389	0.0041583	0.76195	0.75908	0.76117	0.26928
6	0.050265	0.031393	0.0030218	0.77651	0.75397	0.77008	0.31968
7	0.048129	0.031711	0.0036271	0.80726	0.79606	0.79844	0.29605
8	0.044195	0.029955	0.0031003	0.76799	0.72668	0.75594	0.32819
9	0.045253	0.029901	0.0027019	0.77623	0.78187	0.77807	0.30059
10	0.045238	0.030711	0.0032885	0.65056	0.71733	0.64417	0.28253
11	0.044501	0.030233	0.0028072	0.67526	0.66344	0.63756	0.26573
12	0.0435	0.030194	0.0026659	0.73237	0.76329	0.71757	0.29156
13	0.041347	0.029769	0.0022592	0.76863	0.73132	0.72369	0.28687
14	0.042873	0.030436	0.002712	0.81915	0.78572	0.79091	0.32184
15	0.041629	0.028444	0.0019515	0.75728	0.76276	0.71338	0.28462
16	0.041074	0.030665	0.0020052	0.77266	0.72503	0.71151	0.3037
17	0.04006	0.027966	0.0022102	0.77484	0.80502	0.76886	0.32596
18	0.039737	0.030473	0.0018982	0.77683	0.7705	0.74447	0.30875
19	0.039538	0.029306	0.0019425	0.78917	0.77015	0.75389	0.3199
20	0.040034	0.030499	0.0022515	0.82617	0.7877	0.78583	0.33605
21	0.038736	0.02729	0.0018633	0.74598	0.74181	0.72106	0.32372
22	0.039357	0.028591	0.0016688	0.71691	0.75694	0.70182	0.29653
23	0.038528	0.027093	0.0017909	0.80804	0.78135	0.78629	0.35293
24	0.038324	0.028817	0.0018073	0.74582	0.73966	0.7106	0.30731
25	0.037931	0.028031	0.001573	0.79757	0.77744	0.76831	0.32271
26	0.037142	0.02853	0.0018409	0.7863	0.76675	0.73634	0.32102
27	0.03768	0.027799	0.0015768	0.77439	0.78968	0.74644	0.31889
28	0.036029	0.026252	0.0017225	0.73943	0.78371	0.71871	0.30211
29	0.03621	0.026359	0.0013504	0.73293	0.72862	0.66623	0.30135
30	0.036205	0.028631	0.0011803	0.78116	0.77455	0.7264	0.33605
31	0.035961	0.027767	0.001729	0.77113	0.76042	0.73581	0.32773
32	0.035467	0.027608	0.0015895	0.78363	0.80781	0.7634	0.34591
33	0.03514	0.028045	0.0013435	0.77132	0.76529	0.73046	0.32709
34	0.035293	0.027171	0.0015274	0.83233	0.80728	0.80193	0.3464
35	0.035474	0.027315	0.0013182	0.7532	0.73053	0.70035	0.29522
36	0.035284	0.027915	0.0010894	0.83516	0.78628	0.78993	0.34455
37	0.034215	0.024682	0.0013777	0.76866	0.76624	0.72068	0.32487
38	0.034724	0.027097	0.0013185	0.71526	0.7293	0.6643	0.3107
39	0.034988	0.028053	0.0013295	0.79016	0.78876	0.74965	0.33767

图 9 模型训练结果

5.3.2 通过 K-means 算法生成合适的先验框

由于物体的大小往往是有一定比例的，不同距离的物体尺寸遵循一定的特征分布。因此，要使用聚类算法来生成合适的先验框。

我们采用聚类的目的是先验框和真实框有更大的 IOU 值。IOU 指的是先验框和真实框之间的交并比最理想的情况下是 IOU 为 1 时，即先验框和真实框完全重叠，此时两者之间距离最小，聚类效果最好。

k-means 拿到数据里所有的目标框，得到所有的宽和高，在这里面随机取得 9 个随机中心，之后以 9 个点为中心得到 9 个簇，不断计算其他点到中点的距离调整每个点所归属的簇和中心，直到 9 个中心不再变即可。这 9 个中心的 x, y 就是整个数据的 9 个合适的 anchors 框的宽和高。

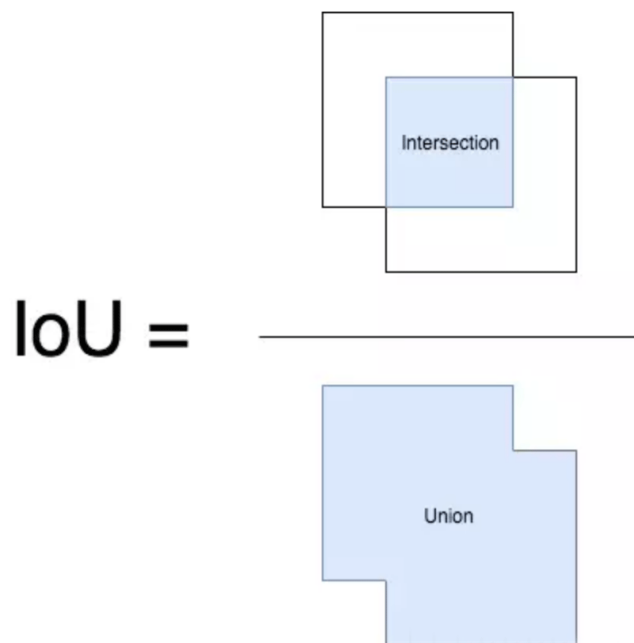


图 10 IOU

5.3.3 模型训练过程分析

- 1) 优化器：使用 Adam，比较常用而且能够快速收敛。整个过程分为两个阶段训练，第一阶段使用较大的学习率进行探索，当第二阶段开始微调时，再换为较小的学习率，所以在 YOLOv5 中是可以设置学习率的。
- 2) 在 callbacks 回调函数中加入 ReduceLROnPlateau 函数，从而来调整学习率，指标停止提升时会减少学习速率。
- 3) epoch 指在训练时数据集被轮几次，对于有限的数据集通过增加 epoch 的次数使得数据集能够在卷积网络中传递多次，模型才能不断被调整。另外 epoch 会和 batch size 同时影响到更新权重的频率。另外需要注意的是，为了避免数据集的有序性对训练产生影响，训练的数据在 train.py 代码中已经将数据随机打乱，并且每个 epoch 训练完后还会再打乱。
- 4) 为了能够实现实时检测，需要调用摄像头，使用 openCV 实现读取摄像头功能，然后在运行时便可以开启摄像头实时检测。

```

def open_cam(self):
    self.webcam_detection_btn.setEnabled(False)
    self.mp4_detection_btn.setEnabled(False)
    self.vid_stop_btn.setEnabled(True)
    self.vid_source = '0'
    self.webcam = True
    th = threading.Thread(target=self.detect_vid)
    th.start()

### 开启视频文件检测事件 ###

def open_mp4(self):
    fileName, fileType = QFileDialog.getOpenFileName(self, 'Choose file', '', '*.mp4 *.avi')
    if fileName:
        self.webcam_detection_btn.setEnabled(False)
        self.mp4_detection_btn.setEnabled(False)
        # self.vid_stop_btn.setEnabled(True)
        self.vid_source = fileName
        self.webcam = False
        th = threading.Thread(target=self.detect_vid)
        th.start()

```

图 11 摄像头实时检测与视频文件检测

5.4 模型训练结果的评估

在训练完后需要对模型进行评估，如下图为模型的混淆矩阵，其中口罩检测精度约为 0.78，人脸检测精度约为 0.96，可以考虑对模型进行一个优化。

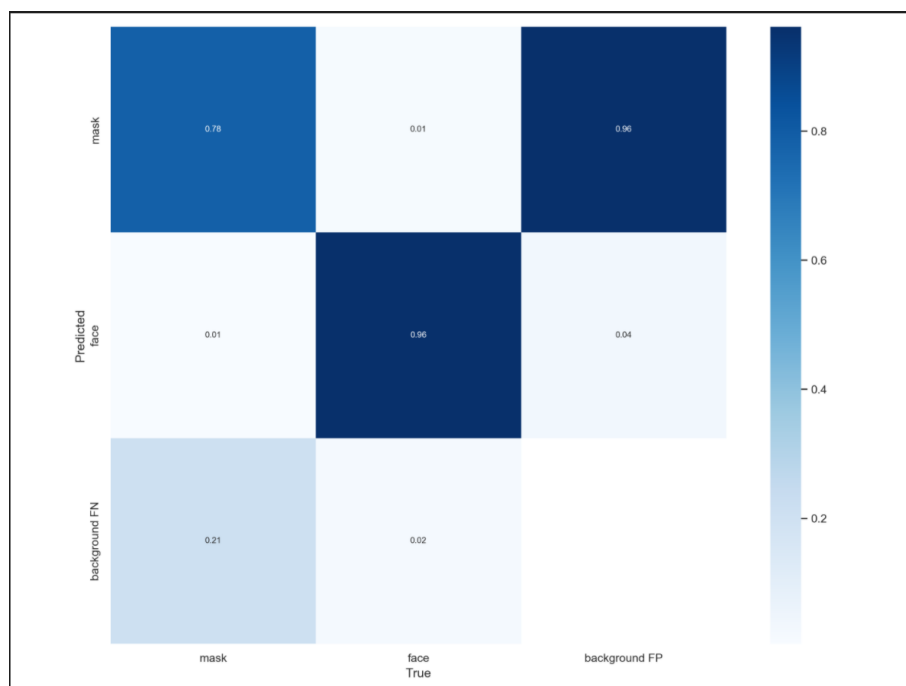


图 12 混淆矩阵

F1 函数是精确率和召回率的调和平均数，最大为 1，最小为 0，可见稳定在 0.79。

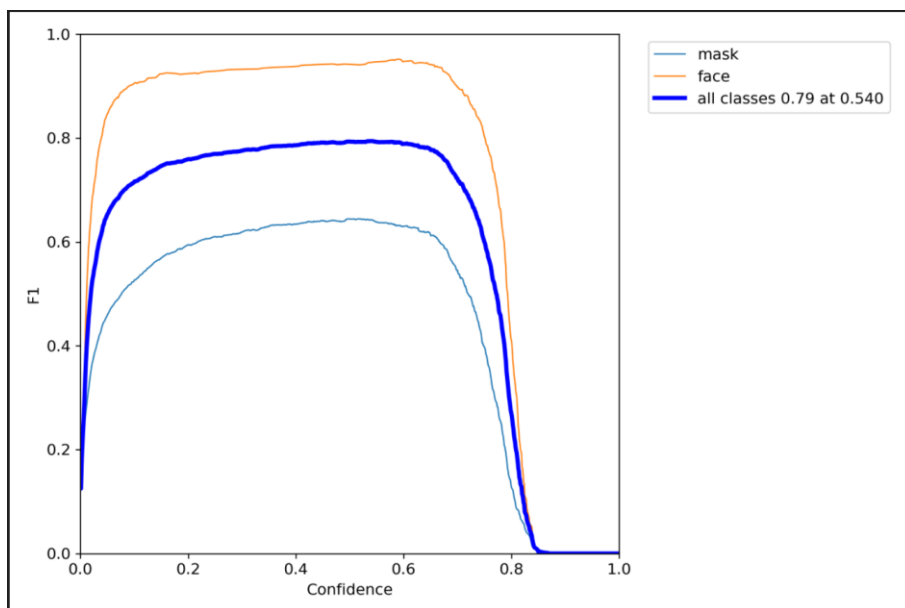


图 13 F1 函数

下图为 PR-curve，可见我们的模型在验证集上的均值平均密度为 0.786。

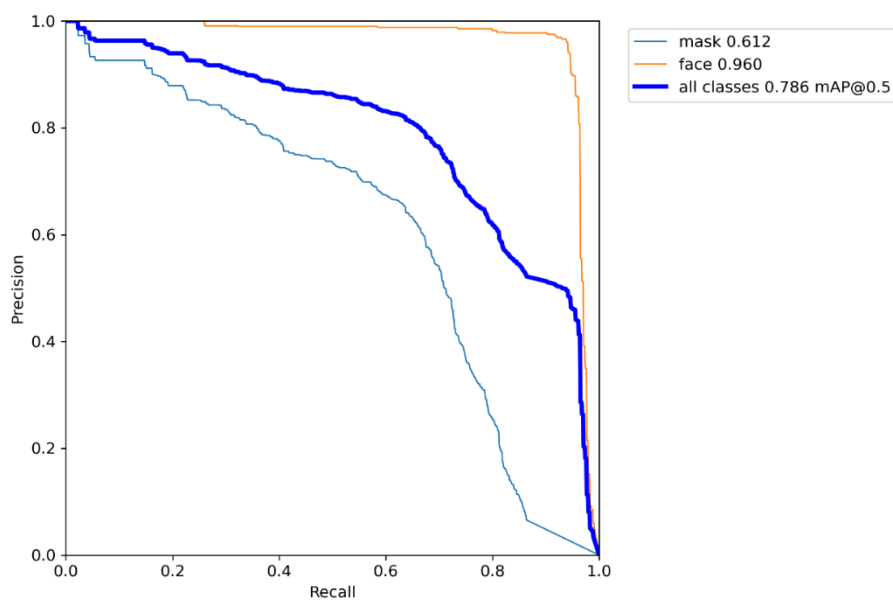


图 14 PR-curve

下图为识别的精度和错误率等随着训练的变化过程

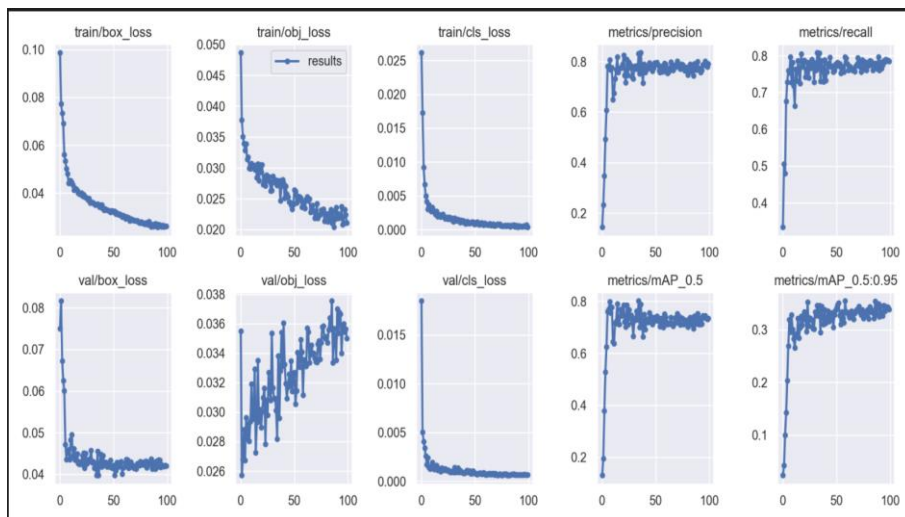


图 15 识别的精度和错误率等随着训练的变化过程

5.5 模型的优化

我们从以下两个方面，对模型进行了优化。

第一方面：我们反复实验先验框的尺寸和数量，在实际训练中通过不断地修改先验框尺寸从而找到恰当的先验框，来加快训练收敛速度。通过反复实验，我们发现学习率为 0.001 的时候效果是比较好的。

第二方面：参数调整，通过控制变量法调整参数，包括学习率，epoch, batch-size 等。另外，我们尝试加入函数 `ReduceLROnPlateau` 来调整学习率，这个函数是用来监测某个指标的，我们通过监测 loss 值，当指标不再提升时，减少学习率。当学习停滞时，减少 2 倍或 10 倍的学习率以此获得更好的效果。

5.6 成果展示

我们使用 `pyqt5` 制作了一个简单的可视化交互界面，从而让用户能够利用我们的系统进行图片检测、实时摄像头检测和视频文件检测这三个功能。

5.6.1 图片识别功能

由下图可见，图中戴口罩的人都被红框圈出，没戴口罩的人被粉色框圈出。识别结果较为准确。



图 16 图片检测

由下图可见，在较为复杂的场景中，口罩检测的效果仍然较好，没有发生误测。



图 4 较为复杂的图片检测

5.6.2 视频和摄像头实时识别功能

由下图可见，通过笔记本摄像头进行口罩实时识别，精准地识别出了戴口罩的对象和没戴口罩的对象，没有发生误测。



图 18 摄像头实时检测

6 总结与展望

6.1.1 总结

通过对比多个模型，我们可知，YOLOv5 在准确率和速度上相对较优秀。我们在确定以 YOLOv5 为基础框架后开始训练模型，选取了合适的先验框加快训练收敛速度，并通过增加迭代次数，调整学习率等方式不断尝试以获取最优的模型。通过最终的实际效果可见，该口罩检测系统的效果和性能较好，准确率较高。

6.1.2 展望

首先在检测十分密集和多角度的场景下，还是会存在漏检情况，所以在人脸识别和先验框选取的算法上还有待改进。

其次，目前我们只实现了通过笔记本调用自身摄像头进行的一个实时检测，希望后续有能力的情况下实现能在移动端和嵌入式系统中的适配与运用。

6.1.3 感想

通过这次课程设计，我们学会了通过 YOLOv5 这一计算机视觉模型来对目标进行检测，并成功将其运用在了一个符合现实背景的场景中，受益匪浅。